

LangSmith Studio - step-by-step setup & debugging guide (with `uv`)

LangSmith Studio - step-by-step setup & debugging guide (with `uv`)

A hands-on walkthrough that takes you from zero to debugging a LangGraph agent in LangSmith Studio, using `uv` for environment and dependency management instead of `conda` + `pip`.

Why? It replaces `conda`, `pip`, and `virtualenv` with one fast Rust binary. creates virtualenvs in milliseconds, resolves and installs an order of magnitude faster than `pip`, and lets you execute scripts without manually activating the env. If you're new to it: <https://docs.astral.sh/uv/>.

?? Run this locally, not on Colab. Studio is a browser app at <https://smith.langchain.com/studio> that connects to a local LangGraph server you start with `uv run langgraph-cli`. Colab is a remote VM with no addressable localhost. The notebook is markdown + shell commands - you don't have to run any Python cells if you'd rather copy-paste into a terminal.

What you'll build

Two projects, in order:

1. `01_hello_world` - a 30-line throwaway. Five minutes from `0` to first run in Studio. Use this to learn the mechanics.
2. `02_structured_output` - port the workshops capstone. Use this to learn the debugging workflow that makes structured-state routing pay off.

Prerequisites

- * Python 3.11+ somewhere on your machine (or let `uv` install it for you - see §0)
- * An OpenAI API key (`sk-...`)
- * A LangSmith API key (`lsm-...`) - free tier is fine; sign up at <https://smith.langchain.com>
- * A terminal you can leave running while you click around in a browser

Part 0 - Install `uv`

One-liner installer. Pick your platform:

Code

```
# macOS / Linux
curl -Lsf https://astral.sh/uv/install.sh | sh
# Windows (PowerShell)
# powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
# Or via Homebrew / pipx if you already have them:
# brew install uv
# pipx install uv
```

Verify:

Code

```
uv --version
# uv 0.5.x or later - anything from late 2024 onward is fine
```

One-time setup. doesnt need a global Python - itll download the right interpreter on demand. If you want to pin one explicitly: .

Part 1 - How Studio actually works

Studio doesnt host your graph. Its a frontend. The graph lives in a Python process you run locally with , and Studio talks to that process over HTTP. Mental model:

When you click Submit in Studio, the browser POSTs to . When you click Re-run from here, it POSTs to a fork endpoint with the edited state. Everything runs in the Python process on your laptop - Studio is just a

fancy GUI for the LangGraph servers HTTP API.

Three consequences worth internalizing:

- * Kill and Studio breaks. The browser tab will show no connection. Restart the server and refresh.
- * In-memory checkpoints dont survive restarts. Saving a code file hot-reloads the graph and wipes threads. For multi-turn debugging across reloads, swap for .
- * Browser ? localhost is allowed by CORS because opens permissive headers. If you see CORS errors, make sure you opened Studio via the URL the CLI printed - that URL has the param baked in.

Part 2 - Project 1: the 5-minute weather agent

We'll scaffold a minimal project, start the server, open Studio, run a few invocations, and time-travel through the trace.

2.1 Create the project layout

Code

```
mkdir -p weather-agent/src
cd weather-agent
```

Final layout:

2.2 Initialize the project with `uv`

creates a , pins the Python version, and seeds a virtualenv folder. We pass and because we dont want sample boilerplate - just an env to install into.

Code

```
uv init --no-readme --no-package --python 3.11 .
```

This drops three files into the current dir:

- * - project metadata + dependency list
- * - the Python version will use here ()
- * - boilerplate sample; delete it:

2.3 Add dependencies with `uv add`

is the equivalent of - it (a) resolves the dep, (b) writes it into , and (c) installs it into the projects virtualenv (creating if it doesnt exist). Its an order of magnitude faster than .

Code

```
uv add \  
"langchain==1.2.15" \  
"langchain-openai==1.1.16" \  
"langchain-core==1.3.0" \  
"langgraph==1.1.9" \  
"langgraph-checkpoint-sqlite==3.0.3" \  
"openai==2.32.0" \  
"pydantic==2.13.3"\  
# The langgraph CLI itself, with the in-memory server extra  
uv add "langgraph-cli[inmem]"
```

A file appears - thats the resolved lockfile (commit it). The folder is also created automatically; dont commit that.

2.4 Create `src/agent.py`

Studio loads this file and looks for the variable named in (well wire that next). The variable must be a compiled LangGraph - i.e. the result of or .

Code

```
%%writefile src/agent.py
from langchain.agents import create_agent
from langchain.tools import tool
@tool
def get_weather(city: str) -> str:
    """Get the weather for a city."""
    return f"Sunny, 19°C in {city}"
graph = create_agent(
    model="gpt-4o-mini",
    tools=[get_weather],
    system_prompt="You are a concise weather assistant.",
)
```

2.5 Create `langgraph.json`

Project manifest. The CLI reads it on startup. Format:

You can register multiple graphs by adding more keys.

Code

```
%%writefile langgraph.json
{
  "dependencies": ["."],
  "graphs": {
    "weather_agent": "./src/agent.py:graph"
  },
  "env": ".env",
  "python_version": "3.11"
}
```

2.6 Create `.env`

Read by the CLI; values become environment variables for your graph. Never commit it.

Code

```
%%writefile .env
OPENAI_API_KEY=sk-replace-me
LANGSMITH_API_KEY=lsv2_replace-me
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=studio-weather-demo
```

2.7 Start the server with `uv run`

automatically uses the projects virtualenv - no manual needed. It also re-syncs deps if changed since the last run.

Code

```
uv run langgraph dev
```

You should see something like:

...

Welcome to LangGraph!

* ? API: <http://127.0.0.1:2024>

* ? Studio: <https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024>

* ? API Docs: <http://127.0.0.1:2024/docs>

Watching for changes

...

Ctrl-click the Studio URL. Browser opens, authenticates against LangSmith using from your , and the graph topology renders.

2.8 Common startup failures

Leave the server running in this terminal for the rest of Part 2.

Part 3 - Tour the Studio UI

Studio has three panels.

3.1 Left panel - Graph

Renders your topology. Boxes are nodes, lines are edges, dashed lines are conditional edges, curves are loop-backs. As a run executes, the active node pulses and edges light up as they're traversed.

What to do here:

- * Sanity-check topology. If you see an unexpected node or a missing edge, your / calls are wrong. Catch this before debugging anything else.

- * Click any node. The centre panel filters to that node's invocations across the run.

3.2 Centre panel - Thread

A vertical timeline of every node execution for the currently-selected thread. Each entry is collapsible and shows:

- * The input state delta the node received
- * The output state delta the node returned
- * Any / content it emitted

Multiple runs on the same thread stack here, separated by checkpoint markers. Multi-turn conversations show as one growing thread; switching from the dropdown swaps the visible thread.

3.3 Right panel - three tabs

The crucial button at the bottom of the State tab: ? Re-run from here. It forks the graph from the selected checkpoint, applies any state edits you've made, and runs forward.

Part 4 - Three workflows you'll use constantly

4.1 Submit a run from scratch

1. Top of centre panel ? New thread (Studio generates a).
2. Right panel ? Inputs tab. For a agent, the minimum input is:
3. Click Submit.
4. Watch the graph panel: ? (the LLM node) ? (calls) ? (final answer) ? .
5. Centre panel populates with each node's I/O. Right-panel State tab now shows the final state.

4.2 Continue the same thread (multi-turn memory)

After §4.1 completes:

1. Stay on the same thread (don't click New thread).
2. Inputs tab ? enter only the new turn:

Dont re-include prior messages - the checkpoint already has them.

3. Submit. The agent runs on top of the existing thread state and remembers Paris was just asked.

This is the GUI equivalent of:

4.3 Time travel - replay or fork from a past step

1. Centre panel ? scroll to a past checkpoint (e.g. just before the second invocation).

2. Click that checkpoints ? menu ? Fork from here. Studio selects it as the cursor.

3. (Optional) Right panel State tab ? edit any field.

4. Click ? Re-run from here.

5. Studio creates a sibling branch and runs forward. The original is preserved.

Use this when you want to ask what would have happened if X had been different at step 5?

Part 5 - Breakpoints (a.k.a. interrupts)

There are three ways to pause a graph for inspection or human approval. All of them surface the same way in Studio: the run pauses at the boundary, the right panel shows a ? Resume button, and you can edit state before resuming.

5.1 Static interrupt - set in the Studio UI

Right-click any node in the graph panel ? Add interrupt before or Add interrupt after. Studio sends a config flag to the server; the next run pauses at that boundary. Fastest way to step through a graph without touching code - useful when you dont yet know which node is misbehaving.

5.2 Compile-time interrupts - global, in code

These pause every run at the named boundaries. Studio surfaces them identically. Useful for HITL pipelines where the pause point is fixed (e.g. always pause before sending the email).

5.3 Dynamic interrupt - `interrupt()` inside a node

For real human-in-the-loop where the payload matters:

```
```python
from langgraph.types import interrupt, Command

def review_gate(state):
 decision = interrupt({
 evidence_count: len(state[evidence]),
 sources: state[sources],
 })
 if decision.startswith(redo):
 return {sub_questions: [decision.split(':', 1)[1]],
 critic_sufficient: False}
 return {}
```

...

When the graph hits this node, Studio shows the dict you passed to in the right panel and waits. Type a value (e.g. or ) and click Resume. The server returns your value as the result of and the node continues.

## Part 6 - Project 2: port the workshop capstone

Stop the previous (Ctrl-C) and create a fresh project.

### 6.1 Initialize the second project

#### Code

```
cd .. # back out of weather-agent/
mkdir -p research-agent/src
cd research-agent
uv init --no-readme --no-package --python 3.11 .
rm main.py
Same dep set
uv add \
"langchain==1.2.15" \
"langchain-openai==1.1.16" \
"langchain-core==1.3.0" \
"langgraph==1.1.9" \
"langgraph-checkpoint-sqlite==3.0.3" \
"openai==2.32.0" \
"pydantic==2.13.3"
uv add "langgraph-cli[inmem]"
```

*? Faster way for repeat setups. Once youve got a known-good and , just them into the new project folder and run . That installs the exact locked versions in one shot - much faster than re-resolving with .*

## 6.2 Create `src/research_agent.py`

This is the workshops capstone (§8.1 - §8.7) flattened into one file. The only addition at the bottom is - Studio looks for the top-level `variable`.

### Code

```

%%writefile src/research_agent.py
import os, time, operator
from typing import Annotated
from typing_extensions import TypedDict
from contextlib import contextmanager
from langchain.chat_models import init_chat_model
from langchain.messages import AnyMessage, HumanMessage, AIMessage, SystemMessage
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import InMemorySaver
from pydantic import BaseModel, Field
?? State schema ???
class ResearchState(TypedDict):
 messages: Annotated[list[AnyMessage], add_messages]
 research_question: str
 sub_questions: Annotated[list[str], operator.add]
 evidence: Annotated[list[dict], operator.add]
 sources: Annotated[list[str], operator.add]
 critic_sufficient: bool
 critic_reasoning: str
 critic_next_query: str
 iterations: int
 max_iterations: int
 error_count: int
 final_answer: str
?? Search backend (local corpus - no Tavily key required) ?????????????
LOCAL_CORPUS = [
 {"title": "LangGraph v1 release notes",
 "url": "docs.langchain.com/oss/python/langgraph/overview",
 "content": "LangGraph 1.0 stabilized the StateGraph API, renamed MemorySaver to "
 "InMemorySaver, and introduced durable Command/interrupt types. The "
 "langgraph.prebuilt.create_react_agent factory was deprecated in favor "
 "of langchain.agents.create_agent."},
 {"title": "create_agent - LangChain reference",
 "url": "docs.langchain.com/oss/python/langchain/agents",
 "content": "create_agent is the canonical entry-point for tool-using agents in "
 "LangChain v1."},
 {"title": "Reducers in LangGraph",
 "url": "docs.langchain.com/oss/python/langgraph/graph-api",
 "content": "A reducer merges a node's returned partial update into state. Default "
 "is overwrite; add_messages appends messages by id."},
 {"title": "Checkpointers and threads",
 "url": "docs.langchain.com/oss/python/langgraph/persistence",
 "content": "A checkpointer makes a graph durable and replayable. Threads isolate "
 "state by thread_id; get_state_history walks every checkpoint."},
 {"title": "LangSmith Studio",
 "url": "docs.langchain.com/oss/python/langgraph/studio",
 "content": "LangSmith Studio is a browser IDE for debugging LangGraph apps. "
 "Visualizes the graph, shows state at every step, lets you edit state "
 "and re-run from any checkpoint."},
 {"title": "Streaming modes",
 "url": "docs.langchain.com/oss/python/langgraph/streaming",
 "content": "stream_mode=values yields full snapshots; updates yields per-node "
 "deltas; messages yields LLM token chunks."},
]
def search(query: str, k: int = 4) -> list[dict]:
 q = set(query.lower().split())
 def score(doc):
 text = (doc["title"] + " " + doc["content"]).lower()
 return sum(1 for w in q if w in text)
 ranked = sorted(LOCAL_CORPUS, key=score, reverse=True)
 return [d for d in ranked if score(d) > 0][:k] or ranked[:k]
?? Pydantic schemas ???

```

? Note about checkpointers in deployed graphs. When you deploy this graph to the LangGraph Platform (LangSmiths managed runtime), remove the `from` - the platform injects its own. The local server happily

*uses whatever you pass.*

## 6.3 `langgraph.json`

### Code

```
%%writefile langgraph.json
{
 "dependencies": ["."],
 "graphs": {
 "research_agent": "./src/research_agent.py:graph"
 },
 "env": ".env",
 "python_version": "3.11"
}
```

## 6.4 `.env`

### Code

```
%%writefile .env
OPENAI_API_KEY=sk-replace-me
LANGSMITH_API_KEY=lsv2_replace-me
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=studio-research-demo
```

## 6.5 Run the server

## Code

```
uv run langgraph dev
```

Open the printed Studio URL.

### 6.6 Submit your first capstone run

Right panel ? Inputs tab ? paste this initial state:

Click Submit. You should see, in the graph panel:

? ? ? (loop back to ) ? ? ?

The terminal running `uv run langgraph dev` prints the per-node timings from the `instrumentation`.

## Part 7 - The killer demo: debugging structured-state routing

This is the payoff of the capstones design. Because `Router` reads only typed state fields (`state`, `messages`) and never parses message content, you can debug the router by editing one boolean.

Try this in Studio:

1. After §6.6 completes, find the centre-panel entry for `graph` at iteration 1.
2. Click it. The right panel switches to that checkpoint.
3. State tab ? find `graph`. Its `graph` (which is why the loop ran another researcher iteration).
4. Toggle it to `graph`.
5. Click ? Re-run from here.
6. Watch the graph panel. The agent skips the second research loop entirely and jumps straight to `graph`.

You just changed the agents routing decision without touching code or prompts. Thats only possible because the router reads typed state.

Compare this to a string-parsing router (`Router`): youd have to find and edit the message, hope your regex still matches, and re-run blindly.

Now try the inverse:

1. Find a checkpoint where `graph` is `graph`.
2. Toggle it to `graph`.
3. Re-run. The agent does another research loop.

The original runs are preserved as sibling branches - Studios tree view lets you flip between them.

## Part 8 - A real failure walkthrough

You'll eventually hit a bug where the agent returns even though the question is simple. Here's the diagnostic playbook in Studio.

Step 1 - Graph panel: topology check. Confirm the edges match your mental model. Missing edge? Unreachable node? Fix that first.

Step 2 - Centre panel: find the anomaly. Scroll the timeline. You see ran twice followed immediately by . Suspicious - searches looked like they succeeded.

Step 3 - Click (iteration 2) ? State tab at its input checkpoint. You see and is also . Iteration 2 is searching for the empty string, returning no hits, treating no hits as failure, bumping .

Step 4 - Click the preceding (iteration 1) ? State tab at its output checkpoint. You see , , and crucially . The critic intended to propose a follow-up but emitted an empty string. The bug is the critics prompt - not the router, not the researcher.

Step 5 - Validate the hypothesis with one click. At that same checkpoint, type into the editor. ? Re-run from here. Now runs with a real query, returns 4 hits, accepts, produces an answer. Confirmed.

Step 6 - Share the trace. Top of the thread ? Share. Studio gives you a URL. Paste into your PR or ticket. Reviewer opens it and sees the exact pathological state without running anything.

Step 7 - Fix the prompt. Edit the critics in to require a non-empty whenever . Save the file. hot-reloads. Re-submit the original input. Bug is gone.

This loop - edit state in Studio ? confirm the hypothesis ? fix the code ? hot-reload - is the workflow Studio is built for. It's faster than print-statement debugging by a factor of 10× on a real graph.

## Part 9 - Smell ? cause cheat sheet

Pattern-match these when scanning a misbehaving run.

## Part 10 - Make checkpoints survive restarts

is fine for the first 10 minutes. For real debugging - where you want to come back to a thread tomorrow - swap it for SQLite. Edit the bottom of :

```
```python
import os
from langgraph.checkpoint.sqlite import SqliteSaver

if os.getenv(LANGGRAPH_DEPLOYED):
    # Platform injects its own checkpointer; dont pass one.
    graph = b.compile()
else:
    # Local dev - persistent SQLite so threads survive hot-reloads and restarts.
```

```
_sqlite_ck = SqliteSaver.from_conn_string(dev.db).enter()
graph = b.compile(checkpointer=_sqlite_ck)
...
```

Now `graph` accumulates checkpoints. Add it to `dev` - it has real conversation data. To start fresh: `dev.reset()`.

? Production option. For Postgres, use `PostgresSaver`. Same pattern. It supports concurrent writers, which `SqliteSaver` does not.

Part 11 - Studio vs other observability tools

Rule of thumb: notebook for development, Studio for debugging a specific run, LangSmith for production observability over time.

Part 12 - `uv` tips & gotchas

12.1 The commands you'll actually use

12.2 Studio-specific gotchas

Part 13 - What to commit, what not to

Add this to `.gitignore` in every Studio project:

Code

```
%%writefile .gitignore
.env
*.db
.langgraph_api/
.venv/
__pycache__/
```

Commit:

- *
- * - your dependency declarations

- * - pinned, reproducible install (the whole point of using)

- *

- *

- * (a redacted template - same keys, dummy values)

Dont commit:

- * - has real keys

- * - has real conversation data

- * - server cache

- * - your virtualenv

Thats everything. The natural workflow:

1. running in one terminal.

2. Editor open on .

3. Studio in a browser tab.

4. Edit code ? save ? Studio hot-reloads ? test ? if broken, click a checkpoint, edit state, Re-run from here ? confirm hypothesis ? fix code ? save ? repeat.

That tight loop is what makes Studio worth wiring up for every serious LangGraph project.